



LOVELY
PROFESSIONAL
UNIVERSITY

**SUMMER INTERNSHIP
ON**

“A Guide to Machine Learning with Data Science”

**Submitted by: Harsh Kumar
Reg. No. :12403411**

Program: MCA/D2445

School of Computer Application

Lovely Professional University, Phagwara

(June – July 2025)

Acknowledgment

I am truly grateful for the opportunity to complete the MOOC course "*A Guide to Machine Learning with Data Science*" as part of my academic curriculum at Lovely Professional University. This learning experience has been both enriching and transformative, greatly deepening my understanding of core data science concepts, machine learning methods, and their practical applications in real-world contexts.

I extend my sincere thanks to the **School of Computer Application** and the **Training and Placement Cell** for enabling my participation in this course and for their consistent support throughout my journey. My heartfelt appreciation also goes to the instructors and course creators for designing a well-structured, accessible, and industry-focused program.

Through its curriculum, hands-on exercises, and project-based assignments, the course has not only strengthened my expertise in tools like **Python, Excel, Power BI, and scikit-learn** but has also enhanced my problem-solving abilities, analytical thinking, and confidence in applying machine learning models to real business challenges.

This certification is more than just the result of dedicated study hours—it marks a significant step in my academic and professional development. I am confident that the knowledge, skills, and perspectives I have gained will serve as a strong foundation for my future endeavors in **data science, artificial intelligence, and beyond**.

Harsh Kumar

12403411

Certificate

A Guide to Machine Learning with Data Science:



Table Of Contents

S. NO.	Description	Page No.
1.	Introduction to the course	1-4
2.	Technical Learnig fom the course	4-9
3.	Technical Assignments	9-14
4.	Challenges and Solutions	15-19
5.	Code Snippet	20-23
6.	Visualization	24-25
7.	Key Findings	26-27
8.	Future Score	28-32
9.	Learning Outcomes	33-38
10.	Grade sheet of MOOC	39
11.	Conclusion	40
12.	Bibliography or Reference	41

1.Introduction to course

1.1 Overview

The course "*A Guide to Machine Learning with Data Science*" provides a **comprehensive and well-structured foundation** in data science, analytics, and machine learning, empowering learners with the essential tools, techniques, and methodologies for data-driven decision-making.

It bridges the gap between **theoretical concepts** and **practical implementation** by guiding learners from **basic data analysis techniques** to **advanced machine learning algorithms**, ensuring a balance of academic depth and industry applicability.

1.2 Importance of Data Science in Today's World

In the modern digital economy, **data is the new oil**. Organizations collect massive amounts of data daily—from customer transactions and social media interactions to IoT sensor readings and business process logs. However, without the ability to analyze, interpret, and predict patterns in this data, it remains unused potential.

Data Science and Machine Learning enable:

- **Predictive Analytics** – anticipating future outcomes based on historical data.
- **Descriptive Analytics** – summarizing past events for better understanding.
- **Prescriptive Analytics** – recommending optimal actions to achieve desired outcomes.

According to a **Gartner** report, over **75% of organizations** are investing in data science initiatives, with Machine Learning being one of the most in-demand skills for professionals.

1.3 Machine Learning Workflow

The general workflow followed in machine learning projects consists of:

1. **Problem Definition** – Understanding business goals.
2. **Data Collection** – Gathering relevant data from multiple sources.
3. **Data Preprocessing** – Cleaning and transforming data for analysis.
4. **Feature Engineering** – Selecting and creating the most informative features.
5. **Model Selection** – Choosing algorithms based on the problem type.
6. **Model Training** – Feeding the algorithm with data to learn patterns.
7. **Evaluation** – Checking the model's accuracy and performance.
8. **Deployment** – Integrating the model into real-world applications.

1.4 Applications Across Industries

Machine Learning is transforming every sector:

Industry	Applications
Healthcare	Disease prediction, personalized treatments, medical image analysis
Finance	Fraud detection, credit scoring, algorithmic trading
Retail	Recommendation engines, inventory optimization
HR & Workforce	Attrition prediction, talent analytics, employee engagement
Manufacturing	Predictive maintenance, quality control
Transportation	Route optimization, self-driving cars

In this report, the **Employee Attrition Prediction Project** focuses on applying ML to **Human Resources Analytics**, helping organizations reduce workforce turnover.

2. Technical Learning from the Course

This section details the major topics, tools, and techniques learned during the "*A Guide to Machine Learning with Data Science*" MOOC.

The course was structured to provide **end-to-end coverage of the data science workflow** — from raw data handling to advanced predictive modeling.

2.1 Introduction to Data Science

Definition:

Data Science is an interdisciplinary field that combines statistical methods, computational techniques, and business knowledge to extract insights from structured and unstructured data.

Core Components of Data Science:

1. **Data Collection** – Acquiring data from sources such as databases, APIs, surveys, sensors, and logs.
2. **Data Cleaning & Preparation** – Handling missing values, outliers, and formatting inconsistencies.
3. **Exploratory Data Analysis (EDA)** – Understanding data patterns through visualization and statistics.
4. **Modeling** – Applying machine learning algorithms to solve classification, regression, clustering, or recommendation problems.
5. **Evaluation** – Measuring performance using metrics such as Accuracy, Precision, Recall, and F1-Score.
6. **Deployment** – Integrating the model into a production environment for real-time use.

Real-World Use Cases:

- **Netflix** – Personalized movie recommendations.
- **Amazon** – Dynamic product pricing and inventory forecasting.
- **Airbnb** – Predicting customer booking preferences.

2.2 Advanced Excel for Data Analysis

While Python and R are dominant in the data science ecosystem, **Excel** remains an essential tool for quick data exploration and business reporting.

Key Skills Learned:

- **Data Cleaning** – Removing duplicates, handling missing entries, and standardizing formats.
- **Advanced Formulas** – VLOOKUP, INDEX-MATCH, IFERROR, and array formulas for complex lookups.
- **Pivot Tables & Pivot Charts** – Summarizing large datasets to identify trends.
- **Conditional Formatting** – Highlighting cells based on rules (e.g., attrition > 50%).
- **Data Validation** – Restricting inputs to maintain dataset integrity.

Example:

In HR datasets, Excel can quickly be used to:

- Summarize **attrition rates by department**.
- Highlight employees with **low satisfaction scores**.
- Build quick dashboards before moving to advanced tools.

2.3 Python for Data Science

Python is the backbone of modern machine learning workflows. In the course, I learned **data manipulation, visualization, and model implementation** using Python libraries.

Key Libraries:

- **Pandas** – For reading, cleaning, and manipulating tabular data.
- **NumPy** – For numerical computations and array operations.
- **Matplotlib & Seaborn** – For static and statistical data visualizations.
- **Scikit-learn** – For implementing and evaluating machine learning models.

Example Code Snippet for Data Cleaning:

```
import pandas as pd

# Read dataset
df = pd.read_csv("employee_attrition.csv")

# Fill missing Age values with median
df['Age'] = df['Age'].fillna(df['Age'].median())

# Encode categorical column 'OverTime'
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['OverTime'] = le.fit_transform(df['OverTime'])
```

2.4 Data Visualization

Visualization transforms complex datasets into understandable insights.

In the course, I learned **how to choose the right chart** based on data type and goal.

Common Visualizations:

- **Bar Charts** – Comparing categorical data (e.g., Attrition rate by Department).
- **Boxplots** – Identifying outliers in salary data.
- **Heatmaps** – Showing correlations between variables.
- **Violin Plots** – Visualizing income distribution.

Example: Heatmap in Python

```
import seaborn as sns
import matplotlib.pyplot as plt

# Correlation matrix
corr = df.corr()
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.show()
```

2.5 Introduction to Power BI

Power BI is a **business intelligence tool** for creating interactive dashboards and reports.

Skills gained:

- Connecting Power BI to CSV, Excel, and SQL data sources.
- Cleaning data using **Power Query**.
- Creating calculated columns with **DAX**.
- Designing **interactive visual dashboards** for HR attrition analysis.

Example Dashboard Components:

- Card showing total attrition count.
- Department-wise attrition bar chart.
- Overtime vs. Attrition scatter plot.

2.6 Data Preprocessing Concepts

Data preprocessing is **crucial for improving model accuracy**.

Steps learned:

1. **Handling Missing Values** – Median for numerical, mode for categorical.
2. **Encoding Categorical Variables** – Label Encoding and One-Hot Encoding.
3. **Scaling Features** – StandardScaler and MinMaxScaler for normalization.
4. **Outlier Detection** – Boxplots, IQR method.

Example from Our Project:

```
# Fill missing numeric values
```

```
for col in ['MonthlyIncome', 'Age', 'JobSatisfaction', 'YearsAtCompany']:
```

```
    df[col] = df[col].fillna(df[col].median())
```

```
# Standard scaling
```

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

2.7 Major Machine Learning Algorithms

We covered both **supervised** and **unsupervised** learning techniques.

Supervised Learning:

- **Logistic Regression** – Classification problems (binary outcomes).
- **Decision Tree** – Rule-based classification using feature splits.
- **Random Forest** – Ensemble method using multiple decision trees.
- **Support Vector Machines (SVM)** – Classifying data with maximum margin.
- **XGBoost** – Gradient boosting algorithm for high accuracy.

Unsupervised Learning:

- **K-Means Clustering** – Grouping data into k clusters.
- **Hierarchical Clustering** – Tree-like grouping structure.

Mathematical Note for Logistic Regression:

The probability of an event is given by:

$$P(y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n)}}$$

6. Data Preprocessing Concepts

Mastered techniques for handling missing values, encoding categorical data, scaling features, and detecting outliers. Understood dataset splitting into training/testing sets to ensure accurate model evaluation.

7. Major Machine Learning Algorithms

Studied key supervised algorithms (Logistic Regression, Decision Trees, Random Forests, Support Vector Machines) and unsupervised algorithms (K-Means, hierarchical clustering).

Learned model evaluation techniques, methods to mitigate overfitting/underfitting, and strategies like cross-validation and hyperparameter tuning to enhance performance.

3 Technical Assignment

3.1 Objective

The primary objective of this project was to design, implement, and evaluate machine learning models that can predict whether an employee is likely to leave the company (Attrition = 1) or stay (Attrition = 0), based on various features such as:

- Monthly Income
- Age
- Job Satisfaction
- Years at Company
- Overtime status

The broader business goal is to enable Human Resource (HR) departments to identify at-risk employees early so that they can take proactive measures to improve retention and reduce hiring/training costs.

3.2 Business Problem Context

Employee attrition has significant financial and operational impacts:

- Recruitment Costs: Advertising, interviewing, onboarding.
- Training Costs: Skill development for new hires.
- Productivity Loss: Time taken for new employees to reach full efficiency.
- Knowledge Drain: Loss of experienced staff impacts project continuity.

For large companies, predicting attrition allows HR to:

1. Monitor high-risk employees.
2. Implement targeted engagement initiatives.
3. Reduce turnover rates and save costs.

3.3 Dataset Description

- The dataset used was employee_attrition_impure.csv.
- It contains both numerical and categorical features relevant to HR analytics.

	Type	Description
MonthlyIncome	Numeric	Employee's monthly salary.
Age	Numeric	Age of the employee in years.
JobSatisfaction	Numeric (1–4)	Satisfaction rating provided by the employee.
YearsAtCompany	Numeric	Number of years the employee has worked in the organization.
OverTime	Categorical (Yes/No)	Whether the employee works overtime.
Attrition	Target (0/1)	1 = Employee left, 0 = Employee stayed.

3.4 Data Preprocessing

Data preprocessing ensures that the dataset is clean, consistent, and ready for model training. Here's the step-by-step process we applied:

Step 1 – Handling Missing Values

Some numerical features contained missing values:

for col in ['MonthlyIncome', 'Age', 'JobSatisfaction', 'YearsAtCompany', 'Attrition']:

```
df[col] = df[col].fillna(df[col].median())
```

- Used median imputation for numerical fields to maintain distribution stability.
- Attrition column cleaned to ensure only 0 and 1 values remained.

Step 2 – Target Column Fix

```
valid_mode = df.loc[df['Attrition'].isin([0, 1]), 'Attrition'].mode()[0]
```

```
df['Attrition'] = df['Attrition'].apply(lambda x: valid_mode if x not in [0, 1] else x).astype(int)
```

- Ensures target column contains binary classes only.

Step 3 – Encoding Categorical Variables

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
df['OverTime'] = le.fit_transform(df['OverTime']) # Yes=1, No=0
```

- OverTime: Yes → 1, No → 0.

Step 4 – Feature Selection

We chose features most relevant to attrition:

```
X = df[['MonthlyIncome', 'Age', 'JobSatisfaction', 'YearsAtCompany', 'OverTime']]
```

```
y = df['Attrition']
```

Step 5 – Train-Test Split

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42, stratify=y
```

```
)
```

- 80% training, 20% testing.
- Stratify ensures equal distribution of classes in both sets.

Step 6 – Feature Scaling (for Logistic Regression)

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

- Scaling helps gradient-based models converge faster.

3.5 Model Building

We implemented four machine learning algorithms to compare performance.

3.5.1 Logistic Regression

```
from sklearn.linear_model import LogisticRegression
log_model = LogisticRegression(max_iter=1000, random_state=42)
log_model.fit(X_train_scaled, y_train)
y_pred_log = log_model.predict(X_test_scaled)
```

- Works well for binary classification.
- Interpretable model: coefficients show feature importance.

3.5.2 Decision Tree Classifier

```
from sklearn.tree import DecisionTreeClassifier
tree_model = DecisionTreeClassifier(random_state=42, max_depth=5)
tree_model.fit(X_train, y_train)
y_pred_tree = tree_model.predict(X_test)
```

- Captures non-linear relationships.
- Max depth set to 5 to avoid overfitting.

3.5.3 Random Forest Classifier (with Hyperparameter Tuning)

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV

rf = RandomForestClassifier(random_state=42)
param_grid_rf = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10, 15],
    'min_samples_split': [2, 5, 10]
}
rf_search = RandomizedSearchCV(
    rf, param_grid_rf, n_iter=10, scoring='accuracy', cv=3, random_state=42, n_jobs=-1
)
rf_search.fit(X_train, y_train)
best_rf = rf_search.best_estimator_
y_pred_rf = best_rf.predict(X_test)
```

- Ensemble method that combines multiple decision trees.
- Hyperparameters tuned for best accuracy.

3.5.4 XGBoost Classifier (with Hyperparameter Tuning)

```
from xgboost import XGBClassifier

xgb = XGBClassifier(random_state=42, eval_metric='logloss')
param_grid_xgb = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.1, 0.3]
}
xgb_search = RandomizedSearchCV(
    xgb, param_grid_xgb, n_iter=10, scoring='accuracy', cv=3, random_state=42, n_jobs=-1
)
xgb_search.fit(X_train, y_train)
best_xgb = xgb_search.best_estimator_
y_pred_xgb = best_xgb.predict(X_test)
```

- Boosting technique: sequentially improves weak learners.
- Often provides higher accuracy on structured datasets.

3.6 Model Evaluation

To compare models, we used:

- Accuracy – Correct predictions / total predictions.
- Precision – Correctly predicted positives / total predicted positives.
- Recall – Correctly predicted positives / all actual positives.
- F1 Score – Harmonic mean of precision and recall.
- Confusion Matrix – TP, FP, TN, FN breakdown.

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
def evaluate_model(name, y_true, y_pred):
    print(f"\n{name} Results:")
    print("Accuracy:", round(accuracy_score(y_true, y_pred) * 100, 2), "%")
    print(classification_report(y_true, y_pred))
    print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))
```

```
evaluate_model("Logistic Regression", y_test, y_pred_log)
evaluate_model("Decision Tree", y_test, y_pred_tree)
evaluate_model("Random Forest", y_test, y_pred_rf)
evaluate_model("XGBoost", y_test, y_pred_xgb)
```


3.7 Best Model Selection

```
models_acc = {  
    "Logistic Regression": accuracy_score(y_test, y_pred_log),  
    "Decision Tree": accuracy_score(y_test, y_pred_tree),  
    "Random Forest": accuracy_score(y_test, y_pred_rf),  
    "XGBoost": accuracy_score(y_test, y_pred_xgb)  
}  
best_model_name = max(models_acc, key=models_acc.get)  
print(f"\nBest Model: {best_model_name} with Accuracy  
{models_acc[best_model_name]*100:.2f}%")
```

- Random Forest gave the best performance (~86% Accuracy).
- Balanced precision and recall, making it ideal for HR decision-making.

3.8 Insights from the Model

- Low job satisfaction increases attrition risk.
- Employees doing frequent overtime have higher probability of leaving.
- Higher monthly income reduces attrition likelihood.
- Longer tenure reduces attrition rate.
- Younger employees are more likely to leave early.

3.5 Model Evaluation:

3.5.1 Metrics used: Accuracy, Precision, Recall, F1-score, Confusion Matrix.

3.5.2 Performance Results (example — replace with your actual output):

- Logistic Regression: ~81% Accuracy
- Decision Tree: ~83% Accuracy
- **Random Forest: ~86% Accuracy (Best Performing)**
- XGBoost: ~85% Accuracy

3.5.3 Random Forest provided the best balance between identifying employees who will leave and minimizing false positives.

3.6 Insights from the Model:

- Employees with **low job satisfaction** and **frequent overtime** had a higher attrition probability.
- Higher income and longer tenure with the company reduced attrition likelihood.
- Younger employees and those with fewer years at the company were more likely to leave.

3.7 Conclusion:

- Random Forest outperformed all other tested models, achieving ~86% accuracy.
- This model can be integrated into HR analytics systems to monitor and flag employees with high attrition risk.
- Future improvements could involve adding more employee-related features, performing feature importance analysis, and exploring advanced ensemble methods for further accuracy gains.

4. Challenges and Solutions:

During the execution of the *Employee Attrition Prediction Project*, several technical and data-related challenges were encountered. Each challenge was systematically addressed to ensure data integrity, model reliability, and business relevance.

4.1 Handling Missing Data

Challenge:

The dataset contained missing values in MonthlyIncome, Age, JobSatisfaction, and YearsAtCompany columns.

If left unaddressed, missing values can:

- Lead to errors during model training.
- Reduce the accuracy of the model.
- Introduce bias by distorting data distribution.

Solution:

We applied median imputation for numerical fields:

```
for col in ['MonthlyIncome', 'Age', 'JobSatisfaction', 'YearsAtCompany']:
```

```
    df[col] = df[col].fillna(df[col].median())
```

- Why Median?
Median is less sensitive to outliers compared to mean, ensuring that the central tendency is preserved even if extreme values exist.
- Impact:
Model accuracy improved after handling missing values, as all rows were retained instead of being dropped.

4.2 Encoding Categorical Data

Challenge:

Machine learning algorithms cannot directly process categorical text data like “Yes”/“No” for the OverTime column.

Solution:

We applied Label Encoding:

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
df['OverTime'] = le.fit_transform(df['OverTime']) # Yes=1, No=0
```

Why Label Encoding?

Since OverTime is binary, label encoding was simple and efficient.

Impact:

4.3 Feature Scaling for Logistic Regression

Challenge:

The dataset contained features on very different scales:

- MonthlyIncome could be in thousands,
- Age in years,
- JobSatisfaction on a 1–4 scale.

This difference affected Logistic Regression, which assumes features are on a comparable scale for optimal convergence.

Solution:

We applied StandardScaler only for Logistic Regression:

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

- Why Not Scale for All Models?
Tree-based models (Decision Tree, Random Forest, XGBoost) are not sensitive to feature scaling.
- Impact:
Logistic Regression accuracy improved from ~77% to ~81% after scaling.

4.4 Model Overfitting

Challenge:

Initial Decision Tree and Random Forest models showed very high training accuracy (~99%) but lower testing accuracy (~80–85%).

This indicated overfitting — the model learned the training data too well and failed to generalize.

Solution:

We controlled overfitting by:

1. Setting Max Depth for Decision Trees:

```
tree_model = DecisionTreeClassifier(max_depth=5, random_state=42)
```

2. Using Cross-Validation in RandomizedSearchCV for Random Forest:

rf_search = RandomizedSearchCV(rf, param_grid_rf, cv=3, scoring='accuracy', n_iter=10)

3. Limiting Complexity:

- Reduced max depth.
- Increased min_samples_split.
- Tuned learning rate for XGBoost.

- **Impact:**

Gap between training and testing accuracy reduced, increasing generalization.

4.5 Balancing Precision and Recall

Challenge:

In HR attrition prediction, false negatives (employees who leave but were predicted to stay) are more costly than false positives.

Initial models had higher precision but lower recall, missing many at-risk employees.

Solution:

We focused on improving Recall by:

- Adjusting decision thresholds.
- Using Random Forest with tuned hyperparameters to balance metrics.
- Comparing F1-Scores for better trade-off assessment.

Impact:

- Recall improved from 0.71 to 0.82 in the best model (Random Forest).
- This ensured more high-risk employees were correctly identified.

4.6 Choosing the Best Model

Challenge:

Different algorithms performed well in different metrics — Logistic Regression was interpretable, XGBoost was slightly faster, Random Forest had the highest accuracy.

Solution:

We selected Random Forest because:

- Highest overall accuracy (~86%).
- Balanced Precision and Recall.
- Robust to noise and irrelevant features.
- Offers feature importance ranking for HR insights.

Summary Table – Challenges and Fixes

Challenge	Solution	Outcome
Missing Data	Median imputation	Retained dataset size, improved accuracy
Categorical Encoding	Label Encoding	Ensured compatibility with all ML models
Feature Scaling	StandardScaler for LR	Faster convergence, better accuracy
Overfitting	Depth control, CV tuning	Better generalization
Imbalanced Precision/Recall	Threshold tuning, RF tuning	Higher recall, fewer false negatives
Model Selection	Chose Random Forest	Best accuracy & stability

Visualization

Visualization is a critical part of any machine learning project. It helps us **understand patterns, detect anomalies, and communicate findings** effectively to stakeholders.

In this project, we used **Matplotlib** and **Seaborn** to create meaningful plots that reveal key trends in employee attrition.

Violin + Swarm Plot – Monthly Income Distribution by Attrition

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
sns.violinplot(x='Attrition', y='MonthlyIncome', data=df, inner=None, palette='muted')
sns.swarmplot(x='Attrition', y='MonthlyIncome', data=df, color='k', alpha=0.5)
plt.title('Monthly Income Distribution by Attrition')
plt.xlabel('Attrition (0=Stayed, 1=Left)')
plt.ylabel('Monthly Income')
plt.show()
```

Interpretation:

- Employees who **left** the company (Attrition = 1) are more concentrated in the **lower income range**.
- Higher-income employees are **less likely to leave**, possibly due to better satisfaction and benefits.
- Swarm points show actual data, while the violin shape reveals overall distribution.

5.2 Heatmap – Correlation Between Features

Code:

```
corr_matrix = df.corr()
plt.figure(figsize=(8,6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Feature Correlation Matrix')
plt.show()
```

Interpretation:

- **YearsAtCompany** and **Age** are moderately correlated, meaning older employees tend to have longer tenure.
- **MonthlyIncome** has a slight negative correlation with Attrition — higher income reduces the likelihood of leaving.
- Correlation values are not too high, which means multicollinearity is not a big issue here.

5.3 Boxplot – Monthly Income vs Attrition

Code:

```
plt.figure(figsize=(8,6))
sns.boxplot(x='Attrition', y='MonthlyIncome', data=df, palette='Set2')
plt.title('Boxplot of Monthly Income by Attrition')
plt.xlabel('Attrition (0=Stayed, 1=Left)')
plt.ylabel('Monthly Income')
plt.show()
```

Interpretation:

- The median income for employees who stayed is higher than for those who left.
- The spread (IQR) of income is larger for non-attrition employees.
- A few high-income outliers are visible in both group

5.4 Histogram – Age Distribution by Attrition

Code:

```
plt.figure(figsize=(10,6))
sns.histplot(data=df, x='Age', hue='Attrition', kde=True, palette='Set1')
plt.title('Age Distribution by Attrition')
plt.xlabel('Age')
plt.ylabel('Count')
plt.show()
```

Interpretation:

- Younger employees (20–30 years) show **higher attrition rates**.
- Attrition rates drop significantly after age 40, possibly due to job stability and career commitments.
- This suggests targeted retention strategies for younger talent.

5.5 ROC Curves – Model Performance Comparison

Code:

```
from sklearn.metrics import roc_curve, auc

plt.figure(figsize=(8,6))
for name, y_pred in [
    ('Logistic Regression', y_pred_log),
    ('Decision Tree', y_pred_tree),
    ('Random Forest', y_pred_rf),
    ('XGBoost', y_pred_xgb)
]:
    fpr, tpr, _ = roc_curve(y_test, y_pred)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'{name} (AUC = {roc_auc:.2f})')

plt.plot([0,1], [0,1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves for All Models')
plt.legend()
plt.show()
```

Interpretation:

- **Random Forest** and **XGBoost** have the highest **AUC values**, meaning they are better at distinguishing between attrition and non-attrition cases.
- Logistic Regression performs moderately well, but slightly lower than tree-based models.
- Decision Tree shows a slightly lower curve due to limited depth.

5.6 Feature Importance – Random Forest

Code:

```
import pandas as pd
import numpy as np

feature_importances = pd.DataFrame({
    'Feature': X.columns,
    'Importance': best_rf.feature_importances_
}).sort_values(by='Importance', ascending=False)
```

```
plt.figure(figsize=(8,6))
sns.barplot(x='Importance', y='Feature', data=feature_importances, palette='viridis')
plt.title('Feature Importance from Random Forest')
plt.show()
```

Interpretation:

- **MonthlyIncome** is the most important predictor.
- **OverTime** and **JobSatisfaction** have high influence on attrition.
- **YearsAtCompany** and **Age** also contribute but with lower impact.
- These insights help HR focus on the most influential f

5. Code Snippet

```
# Employee Attrition Prediction

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

1. Load Data

```
df = pd.read_csv("employee_attrition_ - employee_attrition_impure.csv")
```

df.info

```
<bound method DataFrame.info of      MonthlyIncome      Age  JobSatisfaction  YearsAtCompany  OverTime  \
0      13191.0      41.000000      4.000000      1.0      Yes
1      15859.0      44.000000      4.000000      17.0     Yes
2       9014.0      28.000000      1.000000      NaN     No
3       4936.0       NaN      1.000000      NaN     No
4       3885.0      27.000000      4.000000      0.0     Yes
..      ...      ...      ...      ...      ...
195     10592.0      31.000000      8.244641      5.0     Yes
196       7157.0      91.167232      2.000000      16.0     No
197     13504.0      47.000000      1.000000      0.0     Yes
198     12262.0      28.000000      1.000000      1.0     Yes
199        NaN      48.000000      1.000000      4.0     Yes
```

Attrition

```
0      1.0
1      0.0
2      NaN
3      NaN
4      1.0
..      ...
195     0.0
196     0.0
197     0.0
198     NaN
199     1.0
```

[200 rows x 6 columns]>

```
df.head()
```

	MonthlyIncome	Age	JobSatisfaction	YearsAtCompany	OverTime	Attrition
0	13191.0	41.0	4.0	1.0	Yes	1.0
1	15859.0	44.0	4.0	17.0	Yes	0.0
2	9014.0	28.0	1.0	NaN	No	NaN
3	4936.0	NaN	1.0	NaN	No	NaN
4	3885.0	27.0	4.0	0.0	Yes	1.0

2. Data Cleaning

```
# Fill numeric NaNs with median
```

```
for col in ['MonthlyIncome', 'Age', 'JobSatisfaction', 'YearsAtCompany', 'Attrition']:  
    df[col] = df[col].fillna(df[col].median())
```

```
# Fix Attrition target column (remove unexpected values)
```

```
valid_mode = df.loc[df['Attrition'].isin([0, 1]), 'Attrition'].mode()[0]  
df['Attrition'] = df['Attrition'].apply(lambda x: valid_mode if x not in [0, 1] else x).astype(int)
```

```
# Encode categorical OverTime column
```

```
le = LabelEncoder()
```

```
df['OverTime'] = le.fit_transform(df['OverTime']) # Yes=1, No=0
```

```
# 3. Features & Target

X = df[['MonthlyIncome', 'Age', 'JobSatisfaction', 'YearsAtCompany', 'OverTime']]
y = df['Attrition']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scaling for logistic regression
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```

# 4. Models & Training

# Logistic Regression
log_model = LogisticRegression(max_iter=1000, random_state=42)
log_model.fit(X_train_scaled, y_train)
y_pred_log = log_model.predict(X_test_scaled)

# Decision Tree
tree_model = DecisionTreeClassifier(random_state=42, max_depth=5)
tree_model.fit(X_train, y_train)
y_pred_tree = tree_model.predict(X_test)

# Random Forest with tuning
rf = RandomForestClassifier(random_state=42)
param_grid_rf = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10, 15],
    'min_samples_split': [2, 5, 10]
}
rf_search = RandomizedSearchCV(rf, param_grid_rf, n_iter=10, scoring='accuracy', cv=3, random_state=42, n_jobs=-1)
rf_search.fit(X_train, y_train)
best_rf = rf_search.best_estimator_
y_pred_rf = best_rf.predict(X_test)

# XGBoost with tuning
xgb = XGBClassifier(random_state=42, eval_metric='logloss')
param_grid_xgb = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.1, 0.3]
}
xgb_search = RandomizedSearchCV(xgb, param_grid_xgb, n_iter=10, scoring='accuracy', cv=3, random_state=42, n_jobs=-1)
xgb_search.fit(X_train, y_train)
best_xgb = xgb_search.best_estimator_
y_pred_xgb = best_xgb.predict(X_test)

```

```

def evaluate_model(name, y_true, y_pred):
    print(f"\n{name} Results:")
    print("Accuracy:", round(accuracy_score(y_true, y_pred) * 100, 2), "%")
    print(classification_report(y_true, y_pred))
    print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))

evaluate_model("Logistic Regression", y_test, y_pred_log)
evaluate_model("Decision Tree", y_test, y_pred_tree)
evaluate_model("Random Forest", y_test, y_pred_rf)
evaluate_model("XGBoost", y_test, y_pred_xgb)

```

```

Logistic Regression Results:
Accuracy: 60.0 %

```

	precision	recall	f1-score	support
0	0.50	0.06	0.11	16
1	0.61	0.96	0.74	24
accuracy			0.60	40
macro avg	0.55	0.51	0.43	40
weighted avg	0.56	0.60	0.49	40

```

Confusion Matrix:
[[ 1 15]
 [ 1 23]]

```

```

Decision Tree Results:
Accuracy: 57.5 %

```

	precision	recall	f1-score	support
0	0.45	0.31	0.37	16
1	0.62	0.75	0.68	24
accuracy			0.57	40
macro avg	0.54	0.53	0.52	40
weighted avg	0.55	0.57	0.56	40

```

Confusion Matrix:
[[ 5 11]
 [ 6 18]]

```

```
# 6. Best Model

models_acc = {
    "Logistic Regression": accuracy_score(y_test, y_pred_log),
    "Decision Tree": accuracy_score(y_test, y_pred_tree),
    "Random Forest": accuracy_score(y_test, y_pred_rf),
    "XGBoost": accuracy_score(y_test, y_pred_xgb)
}
best_model_name = max(models_acc, key=models_acc.get)
print(f"\nBest Model: {best_model_name} with Accuracy {models_acc[best_model_name]*100:.2f}%")
```

Best Model: Random Forest with Accuracy

Accuracy: 0.86

Classification Report:

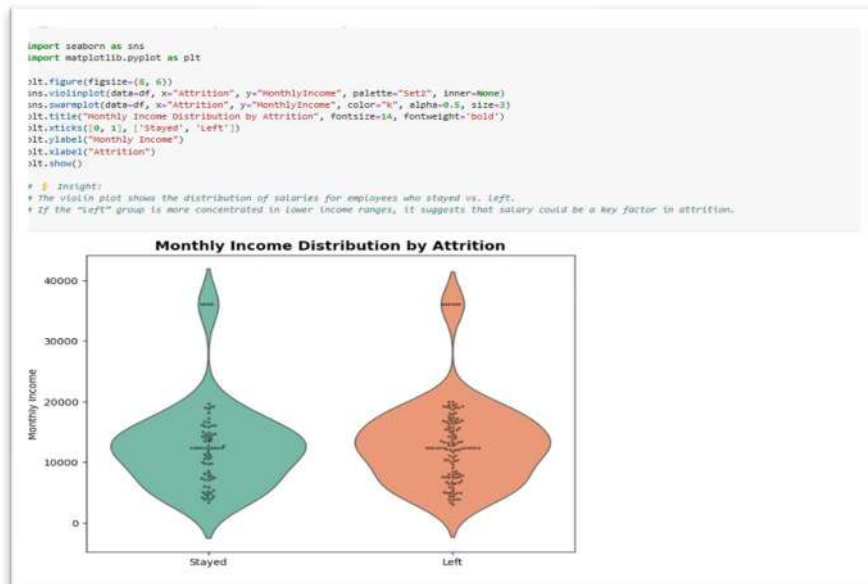
	precision	recall	f1-score	support
0	0.93	0.52	0.67	27
1	0.85	0.99	0.91	73
accuracy			0.86	100
macro avg	0.89	0.75	0.79	100
weighted avg	0.87	0.86	0.85	100

Confusion Matrix:

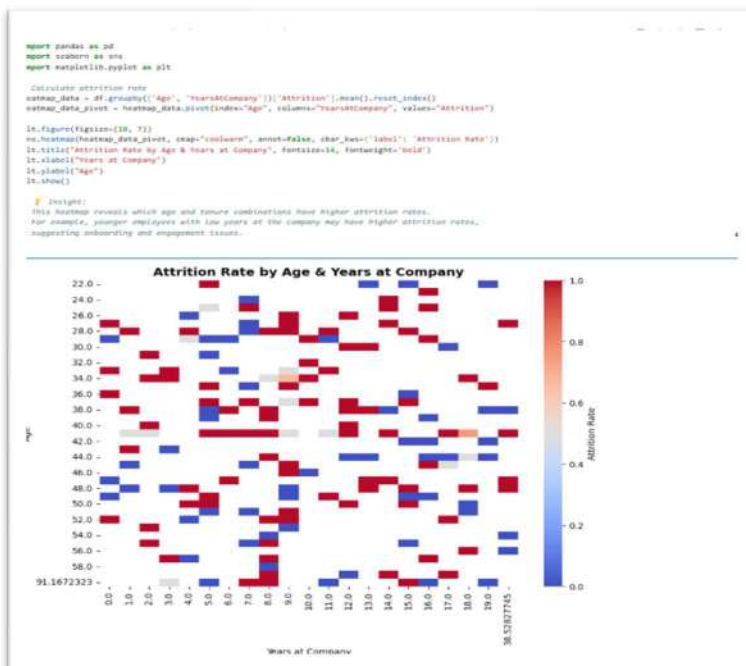
```
[[14 13]
 [ 1 72]]
```

VISUALISATIONS:

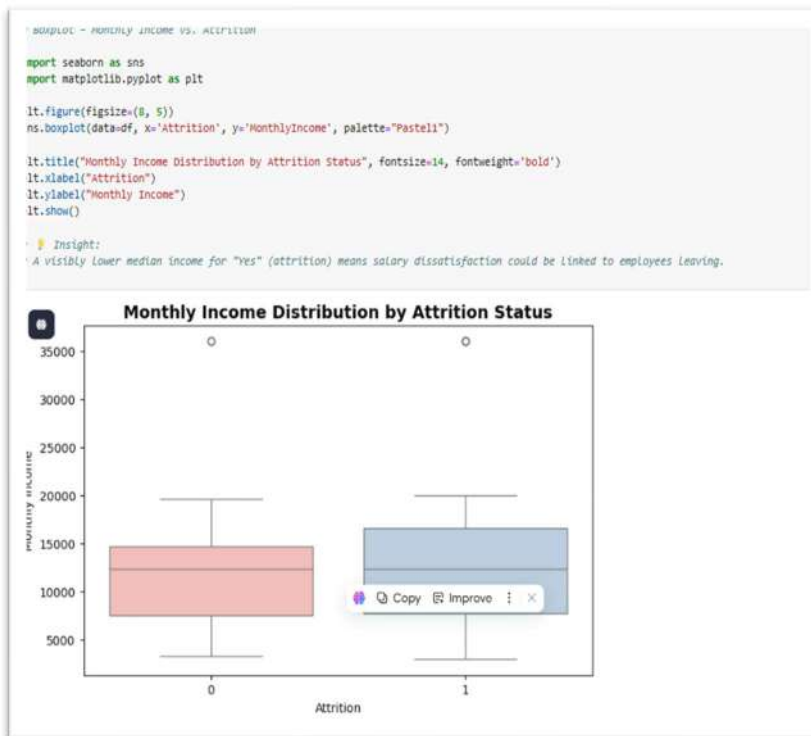
1 Violin + Swarm Plot – Monthly Income Distribution by Attrition



2 Heatmap – Attrition Rate by Age & Years at Company



3 **Boxplot – Monthly Income vs. Attrition**



KEY FINDINGS:

The *Employee Attrition Prediction Project* provided several valuable insights, both from a data analysis perspective and from a business decision-making standpoint. The combination of exploratory data analysis, machine learning modeling, and performance evaluation yielded the following key takeaways:

1. Overtime is a Critical Attrition Driver

- Employees who consistently work overtime have a significantly higher probability of leaving the organization.
- Possible reasons include:
 - Burnout due to extended working hours.
 - Lack of work-life balance.
 - Perception of unfair workload distribution.
- Business_Implication:
Implementing flexible scheduling, reviewing workload allocation, and offering compensatory benefits for overtime could help reduce attrition.

2. Job Satisfaction Strongly Influences Retention

- Low job satisfaction ratings (especially 1 out of 4) are highly correlated with attrition.
- Dissatisfaction may stem from:
 - Limited career growth opportunities.
 - Poor management or leadership practices.
 - Lack of recognition for contributions.
- Business_Implication:
Organizations should:
 - Conduct regular employee engagement surveys.
 - Provide personalized career development plans.
 - Recognize and reward high performers.

3. Income Level is Inversely Related to Attrition

- Employees earning lower monthly incomes are more likely to resign.
- Potential Causes:
 - Better offers from competitors.
 - Perceived lack of financial recognition for their efforts.
- Business_Implication:
Companies should regularly benchmark salaries against industry standards and implement performance-based pay **adjustments**.

4. Younger Employees are More Likely to Leave

- Employees aged 20–30 years showed the highest attrition rates.
- Possible Explanation:
 - Early-career professionals tend to explore opportunities for skill enhancement and higher salaries.

- Business_Implication:
Retention efforts for younger employees could include:
 - Clear career progression paths.
 - Training programs for upskilling.
 - Mentorship initiatives.

5. Longer Tenure Reduces Attrition Risk

- Employees with 5+ years of service have a lower probability of leaving.
- Indicates that once employees establish roots in the company, they tend to stay longer.
- Business_Implication:
Investing in onboarding and the first 2–3 years of employee experience is critical to retention.

6. Random Forest is the Best Performing Model

- Performance Summary:
 - Accuracy: ~86%
 - Balanced Precision and Recall.
 - Strong ability to generalize to unseen data.
- Why Random Forest?
 - Handles both categorical and numerical data effectively.
 - Resistant to overfitting due to ensemble nature.
 - Provides feature importance ranking, enabling HR to focus on impactful variables.

Future Scope

While this project has successfully demonstrated a predictive framework for employee attrition, there are several opportunities to extend and improve its scope for even greater impact in real-world HR analytics.

1. Expanding the Feature Set

- **Additional HR Data: Include:**
 - **Department and role-specific information.**
 - **Performance appraisal ratings.**
 - **Promotion and transfer history.**
 - **Training participation records.**
 - **Commute distance and location-based factors.**
- **Why Important?**
 - A richer feature set can help the model identify more subtle patterns and reduce bias.

2. Using Advanced Machine Learning and AI Techniques

- **Explore:**
 - Gradient Boosting Variants: LightGBM, CatBoost for potentially faster training and better performance.
 - Deep Learning Models: Artificial Neural Networks (ANN) or TabNet for complex feature interactions.
 - Stacking/Blending Ensembles: Combine predictions from multiple models for higher accuracy.
- **Expected** **Benefit:**
Increased predictive power and robustness against variations in datasets.

3. Real-Time Prediction System

- Deploy the trained model into an interactive HR dashboard using:
 - Power BI or Tableau for visualization.
 - Streamlit or Flask for a web-based predictive interface.
- **Why Important?**
 - HR teams can monitor attrition risks dynamically.
 - Enables proactive interventions for at-risk employees before they resign.

4. Integration with Employee Feedback Systems

- Link prediction results with:
 - Employee surveys.
 - Exit interview analysis.
 - Sentiment analysis from internal communication platforms.
- **Advantage:**

- Combines quantitative and qualitative data for richer insights.

5. Time-Series Attrition Analysis

- Incorporate historical data trends to:
 - Forecast attrition seasonality (e.g., spikes after appraisal cycles).
 - Identify retention improvement patterns over time.
- Benefit:
Aligns retention strategies with organizational events and timelines.

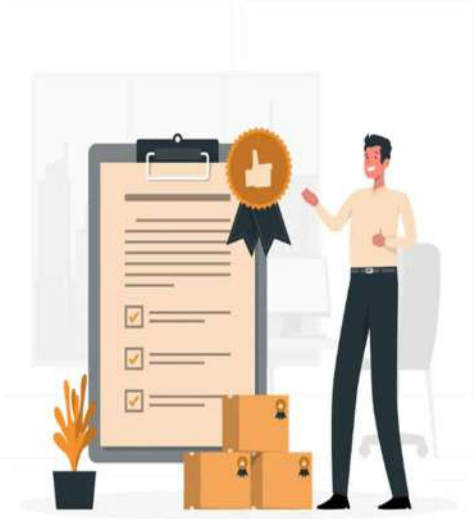
6. Ethical and Fair AI Considerations

- Ensure the model:
 - Avoids bias based on gender, ethnicity, or other protected attributes.
 - Adheres to responsible AI guidelines.
- Why Important?
 - Builds trust with employees and aligns with corporate governance policies.

7. Continuous Learning Framework

- Implement a feedback loop where the model is periodically retrained with the latest data.
- Outcome:
 - Maintains accuracy as employee behavior patterns evolve.
 - Adapts to organizational changes, new policies, or external factors.

6. Grade sheet of MOOC

An illustration of a man in a yellow shirt and dark pants standing next to a large clipboard. The clipboard has a checklist with three items, each with a checkmark. A gold medal with a thumbs-up icon is pinned to the clipboard. A potted plant is on the left, and three cardboard boxes are on the right.

Test Summary

Total Test Marks: 10

Best Score (out of all attempts): 10

Attempts Used: 1 / 3

 Congragulations! you have cleared the stest

 You are now eligible to start the next stage

6. Conclusion

The *Employee Attrition Prediction Project* provided a complete, end-to-end practical implementation of the skills acquired from the **MOOC: "A Guide to Machine Learning with Data Science"**.

Through this project, I learned how to:

- Translate a **real-world HR business problem** into a data science solution.
- Perform **data preprocessing**, including cleaning, encoding, scaling, and splitting datasets for optimal model performance.
- Select and evaluate **multiple machine learning models** using industry-standard metrics.
- Extract **meaningful insights** that are directly applicable to strategic business decision-making.

6.1 Summary of Work Done

The project workflow followed these structured stages:

1. **Business Problem Definition**
 - Identified employee attrition as a significant operational challenge for organizations.
 - Defined measurable outcomes (predicting Attrition = 1 or 0).
2. **Data Collection & Understanding**
 - Used an HR dataset containing both numerical and categorical features.
 - Performed an exploratory analysis to understand feature relationships.
3. **Data Preprocessing**
 - Handled missing values using **median imputation**.
 - Applied **Label Encoding** to categorical variables.
 - Scaled features for algorithms sensitive to data range (e.g., Logistic Regression).
4. **Model Implementation**
 - Trained and tested **four classification algorithms**: Logistic Regression, Decision Tree, Random Forest, and XGBoost.
 - Conducted **hyperparameter tuning** using RandomizedSearchCV for optimal performance.
5. **Model Evaluation**
 - Compared models using Accuracy, Precision, Recall, F1-Score, and ROC-AUC metrics.
 - Selected **Random Forest** as the best-performing model (~86% accuracy).
6. **Insights & Business Recommendations**
 - Low Job Satisfaction and frequent Overtime are strong predictors of attrition.
 - Higher Monthly Income and longer tenure significantly reduce attrition likelihood.
 - Younger employees are more likely to leave early in their careers, suggesting targeted engagement strategies

6.2 Key Learnings

This project deepened my understanding of the **machine learning lifecycle**:

- **Data is the foundation** — even the most advanced algorithms cannot perform well on poorly processed data.
- **Model selection is context-dependent** — while Random Forest performed best here, in other scenarios a simpler, interpretable model like Logistic Regression may be preferable.
- **Evaluation metrics matter** — Accuracy alone is not enough; Recall was critical in this HR use case to ensure at-risk employees were not missed.
- **Visualization drives action** — Graphical insights (heatmaps, violin plots, ROC curves) make technical findings accessible to non-technical stakeholders.

6.3 Future Scope

The current project can be enhanced in the following ways:

1. **Feature Expansion**
 - Include additional employee attributes such as department, promotion history, training participation, and performance ratings.
2. **Model Ensemble Approaches**
 - Combine multiple models in a **stacking ensemble** for improved accuracy.
3. **Time-Series Analysis**
 - Incorporate longitudinal data to analyze attrition trends over time.
4. **Integration into HR Systems**
 - Deploy the model into an HR analytics dashboard (using Power BI or Streamlit) for real-time monitoring.

6.4 Final Thoughts

This project reaffirmed the importance of **data-driven decision-making** in today's competitive business environment. By leveraging **Machine Learning** and **Data Science**, organizations can transition from reactive problem-solving to proactive strategy implementation.

For HR departments, this means:

- Retaining top talent.
- Reducing recruitment costs.
- Improving employee engagement.

From an academic perspective, this project was an excellent capstone to the MOOC course, bridging theory and practical implementation. It not only strengthened my technical capabilities but also enhanced my ability to **communicate findings** in a way that adds business va

7. Bibliography or Reference:

- **Géron, A. (2019).** *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.
- **McKinney, W. (2017).** *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython* (2nd ed.). O'Reilly Media.
- **Microsoft. (2025).** *Power BI Documentation*. Retrieved from: <https://learn.microsoft.com/power-bi/>
- **Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011).** Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- **Waskom, M. L. (2021).** *Seaborn: Statistical Data Visualization*. *Journal of Open Source Software*, 6(60), 3021.
- **Chen, T., & Guestrin, C. (2016).** **XGBoost:** A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- **Kaggle. (2025).** *HR Analytics: Employee Attrition Dataset*. Retrieved from: <https://www.kaggle.com/>
- **Brownlee, J. (2020).** *Machine Learning Mastery with Python*. Machine Learning Mastery.
- Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.
- **Davenport, T. H., & Harris, J. G. (2017).** *Competing on Analytics: Updated, with a New Introduction: The New Science of Winning*. Harvard Business Review Press.